



Actividad 1

Programación Orientada a Objetos Avanzada

Entrega

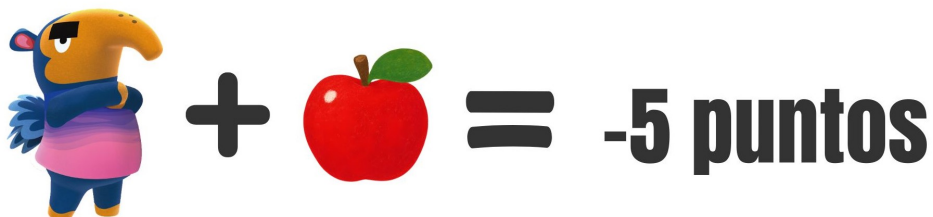
- **Lugar:** Repositorio personal de GitHub — Carpeta: Actividades/AC1
- **Fecha máxima de entrega:** 27 de Agosto 17:10
- **Ejecución de actividad:** La Actividad será ejecutada **únicamente** desde la terminal del computador. Los *paths* relativos utilizados en la Actividad deben ser coherentes con esta instrucción, y no pueden modificarse.

Importante: Antes de comenzar, comprueba que Git este funcionando correctamente en tu repositorio privado. Para esto, **sube los archivos base de la actividad de inmediato** (*add*, *commit*, *push*). Se espera que en esta actividad (así como en las demás actividades y tareas) utilices Git a lo largo de **todo tu desarrollo** como una herramienta, no sólo como un método de entrega. Es por esto que recomendamos enfáticamente que vayas subiendo tus cambios constantemente (*push*), ya que **problemas de último minuto** relacionados con la entrega y Git **no serán considerados**.

Introducción

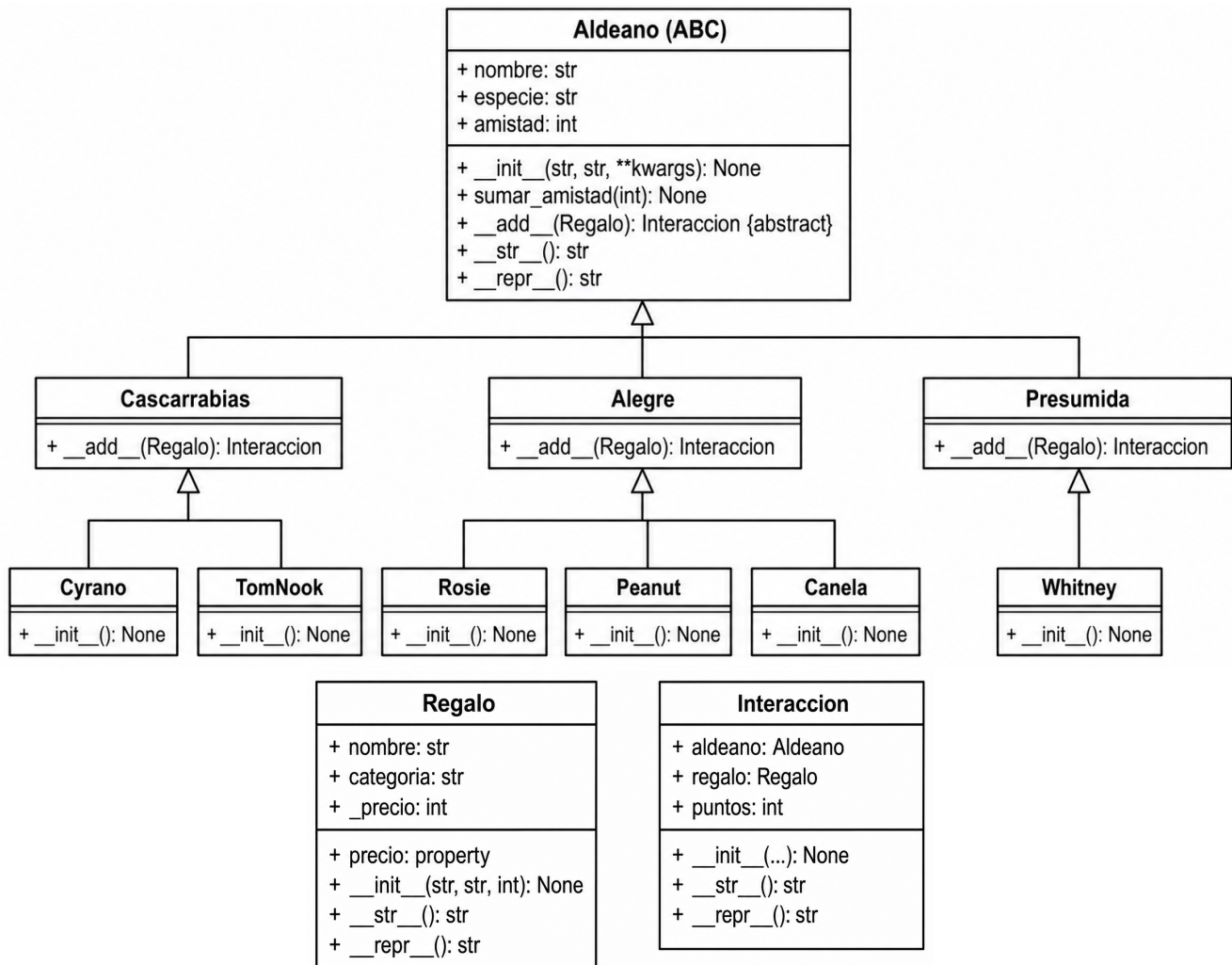
Estás investigando el mundo de *Animal Crossing* y te das cuenta de que las interacciones entre sus aldeanos siguen patrones distintos. Por ejemplo, regalarle una manzana a la **aldeana Rosie** la deja feliz todo el día, mientras que entregarle la misma manzana al **aldeano Cyrano** solo consigue que te mire mal y se vaya. La amistad con cada aldeano puede aumentar o disminuir dependiendo del regalo recibido, ya que cada personalidad valora cosas diferentes.

Para estudiar mejor estas interacciones, decides desarrollar un programa que modele a los **aldeanos**, los **regalos** y la dinámica que ocurre cada vez que un aldeano recibe un objeto.



Diseño del programa

Para este ejercicio no partes desde cero; ya se encuentra estructurado el siguiente modelo de clases inicial:



Esta estructura de clases permite definir el comportamiento general de los aldeanos en la clase abstracta **Aldeano**; las distintas formas de reaccionar ante un regalo en las clases de personalidad **Cascarrabias**, **Alegre** y **Presumida**; y las características particulares de cada aldeano en sus respectivas clases.

Adicionalmente, se definen las clases **Regalo** e **Interaccion** para modelar los objetos que pueden entregarse y registrar el resultado de cada interacción.

Archivos

En el directorio de la actividad encontrarás lo siguiente:

- **No modificar** `bases.py`: Contiene las clases `Aldeano` e `Interaccion`. No debes modificar nada en este archivo, pero se recomienda que leas el código, ya que las clases que debes modificar heredan de estas.
- **Modificar** **Entregar** `regalo.py`: Contiene las clases `Regalo`.
Trabajarás en este archivo en la Parte 1 de esta actividad.
- **Modificar** **Entregar** `aldeanos.py`: contiene las clases que representan a cada aldeano.
Trabajarás en este archivo en la Parte 2 de esta actividad.
- **Modificar** **Entregar** `personalidades.py`: contiene las clases `Cascarrabias`, `Alegre` y `Presumida`.
Trabajarás en este archivo en la Parte 3 de esta actividad.
- **Modificar** `main.py`: puedes modificar todo lo que hay después de `if __name__ == "__main__":` para probar tu código.

Modelo de datos

En esta sección se describen en detalle las clases del archivo `bases.py`.

- **No modificar** `class Aldeano(ABC)`:
 - `def __init__(self, nombre: str, especie: str, **kwargs) -> None:`
Este es el inicializador de la clase, y asigna los siguientes atributos:
 - `self.nombre` Indica el nombre del aldeano.
 - `self.especie` Indica la especie a la que pertenece el aldeano.
 - `self.amistad` Representa el nivel de amistad con el aldeano. Este atributo comienza con un valor de 0.Además, el método `__init__` hace una llamada a `super().__init__(**kwargs)`.
 - `def sumar_amistad(self, puntos: int) -> None:`
Modifica el atributo `amistad` sumando la cantidad indicada por `puntos`. El valor de amistad siempre debe mantenerse dentro del rango entre 0 y 100, ambos inclusive.
El parámetro `puntos` puede ser positivo o negativo, por lo que este método permite tanto aumentar como disminuir la amistad con el aldeano.

- `def __add__(self, regalo) -> Interaccion:`

Es un método abstracto¹.

Define el comportamiento de la clase al utilizar el operador “+” entre un aldeano y un regalo. Como es un método abstracto, debe ser implementado en las subclases de **Aldeano**.

Este método debe retornar una instancia de **Interaccion** que represente lo ocurrido al entregarle el regalo al aldeano.

- `def __str__(self) -> str:`

Retorna una representación del aldeano pensada para ser mostrada al usuario. Esta contiene su nombre, especie y nivel actual de amistad, siguiendo el formato:

nombre (especie) - amistad: amistad

Por ejemplo, si un aldeano llamado Rosie es de especie gato y tiene 20 puntos de amistad, se debe obtener:

Rosie (gato) - amistad: 20

- `def __repr__(self) -> str:`

Retorna una representación del aldeano pensada para quien está programando. Esta incluye el nombre de la clase concreta del aldeano junto con sus atributos **nombre**, **especie** y **amistad**.

■ No modificar `class Interaccion:`

- `def __init__(self, aldeano: Aldeano, regalo, puntos: int) -> None:`

Este es el inicializador de la clase, y asigna los siguientes atributos:

self.aldeano Guarda una referencia al aldeano que recibió el regalo.

self.regalo Guarda una referencia al regalo que recibió el aldeano.

self.puntos Indica la cantidad de puntos de amistad asociados al regalo según la personalidad del aldeano.

- `def __str__(self) -> str:`

Retorna una representación de la interacción pensada para ser mostrada al usuario. Esta indica el nombre del aldeano, el nombre del regalo recibido y la cantidad de puntos de amistad obtenidos o perdidos.

Los puntos se muestran incluyendo explícitamente su signo. Por ejemplo, una interacción que aumente la amistad en 10 puntos se mostrará utilizando +10, mientras que una que disminuya la amistad en 5 puntos se mostrará utilizando -5.

- `def __repr__(self) -> str:`

Retorna una representación de la interacción pensada para quien está programando. Esta incluye el nombre del aldeano, el nombre del regalo y la cantidad de puntos asociados a la interacción.

¹Para indicar que un método es “abstracto” se utiliza el decorador “`@abstractmethod`”.

Parte 1. Los regalos (`regalo.py`)

En esta parte de la actividad debes completar la clase `Regalo`, encargada de representar los distintos objetos que pueden entregarse a los aldeanos del pueblo. Cada regalo tendrá un nombre, una categoría y un precio.

1.1 Inicialización

Al crear un regalo se recibirá la siguiente información:

- **nombre:** nombre del regalo, por ejemplo "Manzana".
- **categoria:** categoría a la que pertenece el objeto. Puede ser "fruta", "ropa" o "mueble".
- **precio:** valor del objeto. No debe modificarse directamente desde fuera de la clase.

1.2 Precio de un regalo

El precio de un regalo debe mantenerse protegido, por lo que su valor interno no debe modificarse directamente desde fuera de la clase. Aun así, se espera que sea posible consultar y asignar el precio utilizando un atributo público, de manera que la clase pueda controlar cualquier cambio realizado sobre este valor.

Al intentar modificar el precio, se debe verificar que este sea válido.

Un precio se considera válido si se encuentra entre 0 y `PRECIO_MAXIMO` campanas², ambos valores incluidos. La constante `PRECIO_MAXIMO` ya se encuentra definida en el archivo.

Si se intenta asignar un valor fuera de este rango, el regalo debe conservar el precio que tenía anteriormente y se debe imprimir exactamente el siguiente mensaje:

```
[Aviso] {valor_recibido} no es un precio valido (0 a {PRECIO_MAXIMO}).  
Se mantiene {precio_actual}.
```

Este mismo comportamiento debe cumplirse cuando el precio recibido al crear un regalo no sea válido, puedes por ejemplo asignarle el precio mínimo válido.

1.3 Representación de Regalo

Durante el desarrollo del programa será útil visualizar la información de los regalos de distintas formas. Al mostrar un regalo directamente al usuario, se espera una representación sencilla que indique su nombre y precio. Por ejemplo, para una manzana de 250 campanas se debería observar:

```
Manzana (250)
```

Por otra parte, al inspeccionar un regalo durante la ejecución del programa, por ejemplo cuando este forma parte de una lista, se espera una representación más detallada que permita identificar su clase y los valores de sus atributos. Para el mismo regalo anterior, esta representación debería ser:

```
Regalo(nombre='Manzana', categoria='fruta', precio=250)
```

Considera las herramientas provistas por Python para controlar cómo se representan las instancias de una clase en cada uno de estos contextos.

²Moneda oficial del mundo de Animal Crossing.

Parte 2. Aldeanos (`aldeanos.py`)

Ahora debes completar la inicialización de cada una de las clases que representan a los aldeanos del pueblo. Cada aldeano debe quedar asociado a su personalidad correspondiente y almacenar correctamente su nombre y especie.

La clase `Cyrano` ya se encuentra implementada como ejemplo. Las características que debe tener cada aldeano son las siguientes:

Clase	Hereda de	nombre	especie
<code>Cyrano</code>	<code>Cascarrabias</code>	"Cyrano"	"oso hormiguero"
<code>Rosie</code>	<code>Alegre</code>	"Rosie"	"gato"
<code>Peanut</code>	<code>Alegre</code>	"Peanut"	"ardilla"
<code>Whitney</code>	<code>Presumida</code>	"Whitney"	"lobo"
<code>TomNook</code>	<code>Cascarrabias</code>	"Tom Nook"	"mapache"
<code>Canela</code>	<code>Alegre</code>	"Canela"	"perro"

Para completar esta parte, considera la relación de herencia entre las clases y reutiliza la lógica de inicialización definida en sus clases padre. Los atributos asociados al nombre y la especie ya son manejados por la jerarquía de clases, por lo que no corresponde redefinir su comportamiento en cada aldeano.

Parte 3. Las personalidades (`personalidades.py`)

Completa el comportamiento asociado a cada una de las personalidades al momento de recibir un regalo. Este comportamiento se define mediante el operador `+`, de modo que una expresión de la forma:

`aldeano + regalo`

genere una interacción y modifique los puntos de amistad del aldeano según corresponda.

Cada personalidad evalúa los regalos utilizando criterios distintos, que pueden depender de su **precio**, su **categoría** o de ambos.

Para esta actividad, todo regalo se clasifica según su precio de una de las siguientes dos formas:

- Un regalo es **caro** si su precio es mayor o igual a `PRECIO_CARO`.
- Un regalo es **barato** si su precio es menor a `PRECIO_CARO`.

La constante `PRECIO_CARO` ya se encuentra definida en el archivo.

Cascarrabias

A los aldeanos con personalidad `Cascarrabias` solo les importa el precio del regalo, independientemente de su categoría.

Caso	Puntos
Regalo caro	10
Regalo barato	-5

Alegre

Los aldeanos con personalidad `Alegre` valoran positivamente cualquier regalo, aunque tienen una preferencia especial por aquellos cuya categoría es `"fruta"`. En este caso, el precio del regalo no afecta los puntos obtenidos.

Caso	Puntos
Regalo de categoría "fruta"	12
Regalo de una categoría distinta de "fruta"	8

Presumida

Los aldeanos con personalidad **Presumida** tienen una preferencia particular por los regalos cuya categoría es "ropa". Si reciben ropa, los puntos dependen además de si el regalo es caro o barato. Para cualquier otra categoría, el precio no afecta los puntos obtenidos.

Caso	Puntos
Regalo de categoría "ropa" y caro	15
Regalo de categoría "ropa" y barato	6
Regalo de una categoría distinta de "ropa"	1

El resultado de la interacción debe reflejarse en la amistad acumulada del aldeano y generar una instancia de **Interaccion** que represente lo ocurrido. Recuerda reutilizar los métodos disponibles en las clases base y considerar que la amistad siempre debe mantenerse dentro del rango permitido por el modelo.

Por ejemplo, algunas interacciones deberían producir los siguientes resultados:

```

Cyrano + Manzana (250, fruta)      -> -5 puntos
Cyrano + Vestido de gala (8600, ropa) -> +10 puntos
Rosie  + Manzana (250, fruta)      -> +12 puntos
Rosie  + Vestido de gala (8600, ropa) -> +8 puntos
Whitney + Manzana (250, fruta)      -> +1 puntos
Whitney + Camisa a rayas (460, ropa) -> +6 puntos
Whitney + Vestido de gala (8600, ropa) -> +15 puntos

```

Output esperado (main.py)

Esta actividad no cuenta con un *output* esperado único. En cambio, se espera que el programa completo funcione de acuerdo con el comportamiento y las especificaciones descritas a lo largo del enunciado.

Para ayudarte a verificar tu implementación, en el archivo `main.py` se incluye un ejemplo de cómo probar algunas de las funcionalidades solicitadas. Estas pruebas son solo un punto de partida: puedes modificarlas, eliminarlas o agregar todas las que consideres necesarias para comprobar que tu código funciona correctamente.

Puntaje por sección

La distribución de puntaje de la tarea es la siguiente. Los criterios que se presentan a continuación buscan indicar los aspectos principales que serán evaluados y pueden subdividirse en más ítems durante la corrección.

■ Parte 1. Regalo: 1,5 pts

- 0,3 pts: Inicializa correctamente los atributos de un **Regalo**, respetando las condiciones indicadas en el enunciado.
- 0,8 pts: Maneja correctamente el precio de un regalo, permitiendo consultarlo y modificarlo desde fuera de la clase, manteniendo su almacenamiento interno protegido y aplicando las validaciones correspondientes.
- 0,4 pts: Implementa correctamente las representaciones de un **Regalo**, mostrando la información solicitada tanto al visualizar el objeto directamente como al inspeccionarlo durante la ejecución del programa.

■ Parte 2. Aldeanos: 1,0 pts

- 0,6 pts: Inicializa correctamente las clases **Rosie**, **Peanut**, **Whitney**, **TomNook** y **Canela**, utilizando los nombres, especies y personalidades correspondientes.
- 0,4 pts: Reutiliza correctamente el comportamiento definido en las clases padre, evitando redefinir innecesariamente atributos o lógica ya disponible.

■ Parte 3. Personalidades: 2,0 pts

- 0,4 pts: Implementa correctamente el comportamiento de **Cascarrabias** al recibir un regalo.
- 0,4 pts: Implementa correctamente el comportamiento de **Alegre** al recibir un regalo.
- 0,5 pts: Implementa correctamente el comportamiento de **Presumida** al recibir un regalo.
- 0,7 pts: Las distintas personalidades integran correctamente este comportamiento con el resto del modelo, actualizando la amistad del aldeano y generando la **Interaccion** correspondiente.

■ Calidad de código y buenas prácticas: 0,5 pts

- 0,3 pts: El código respeta adecuadamente las convenciones de estilo de PEP 8, incluyendo indentación, espaciado y convenciones de nombres.
- 0,2 pts: El código es claro, legible y utiliza buenas prácticas de programación, evitando redundancia innecesaria y aprovechando apropiadamente las herramientas provistas por las clases del programa.

■ Puntaje total: 5,0 pts

Notas

- No puedes hacer *import* de otras librerías externas a las entregadas en el archivo.
- Tu código debe poder ejecutarse por consola como `python main.py` (O variando `python` por `python3`).
- La entrega se realiza subiendo los archivos `regalo.py`, `personalidades.py` y `aldeanos.py` a la carpeta **Actividades/AC1** de tu repositorio personal. Recomendamos no subir otros archivos.
- Recuerda que la ubicación de tu entrega es en **tu repositorio de Git**. En la rama (*branch*) por defecto del repositorio: **main**.
- Siéntete libre de agregar nuevos `print` en cualquier lugar para revisar objetos, modificaciones, etc... Es una herramienta muy útil para encontrar errores. Y si aparece un error inesperado, ¡léelo! Intenta interpretarlo.

Objetivos de la actividad

- Implementar clases siguiendo especificaciones técnicas.
- Manejar concepto de herencia, multiherencia y clases abstractas correctamente.
- Utilizar métodos de clases padres correctamente.
- Utilizar decoradores correctamente.